Cahier des charges

Projet : Dashboard Client & Dashboard Interne – Duality Agency

---

1. Contexte et objectifs du projet

Duality Agency souhaite développer une application web moderne composée de deux tableaux de bord : un **Dashboard Client** destiné aux clients de l'agence, et un **Dashboard Interne** destiné à l'équipe (administrateurs, chefs de projet, développeurs, designers, etc.)

Objectifs principaux

- Améliorer la gestion des projets
- Centraliser la communication client ↔️ équipe
- Optimiser le suivi des tâches et des livrables
- Augmenter la productivité et la transparence

---

2. Périmètre du projet

Inclus

- Application web full stack
- Authentification et gestion des rôles
- Dashboards client et interne
- Backend API sécurisé
- Base de données relationnelle
- Déploiement cloud sur AWS

Exclus (phase 1)

- Application mobile native
- Facturation automatisée (optionnelle en phase 2)
- IA / automatisation avancée

---

3. Utilisateurs & rôles

3.1 Rôles

- **Admin** : gestion globale (utilisateurs, projets, paramètres)
- **Chef de projet** : gestion des projets et équipes
- **Membre équipe** : gestion des tâches assignées
- **Client** : consultation et communication

---

4. Fonctionnalités détaillées

4.1 Authentification & sécurité

- Inscription / connexion
- JWT + Refresh Token
- Gestion des rôles (RBAC)
- Réinitialisation du mot de passe
- Logs de connexion

4.2 Dashboard Client

- Vue d'ensemble des projets
- Statut du projet (En cours, Terminé, En attente)
- Timeline / avancement
- Livrables (upload / download)
- Messagerie avec l'équipe
- Notifications (email + in-app)

4.3 Dashboard Interne

Gestion des projets

- Création / modification / suppression de projets
- Affectation clients & équipes
- Définition des deadlines

Gestion des tâches

- Kanban (To Do / In Progress / Done)
- Priorité des tâches
- Assignation
- Commentaires internes

Gestion des utilisateurs

- CRUD utilisateurs
- Attribution des rôles

Reporting

- Avancement des projets
- Charge de travail par membre
- Historique des actions

---

5 . Architecture technique

5 . 1 Frontend

- **Framework** : Next.js (React)
- **Langage** : TypeScript
- **UI** : TailwindCSS / Shadcn UI
- **State management** : Redux Toolkit / Zustand
- **Auth** : NextAuth ou custom JWT

---

5 . 2 Backend

- **Framework** : NestJS
- **Architecture** : Clean Architecture / Modular
- **API** : REST (Swagger intégré)
- **Sécurité** : JWT, Guards, Validation Pipes

---

5 . 3 Base de données

- **SGBD** : PostgreSQL
- **ORM** : Prisma ou TypeORM

Entités principales

- User
- Role
- Client
- Project
- Task
- Message
- Notification
- File

---

6 . DevOps & AWS

6 . 1 Infrastructure AWS

- **EC** 2 : Backend NestJS
- **RDS (PostgreSQL)** : Base de données
- **S** 3 : Stockage des fichiers
- **CloudFront** : CDN
- **Route** 5 3 : DNS
- **IAM** : Gestion des accès

6.2 CI/CD

- GitHub Actions
- Pipeline :
- Lint & tests
- Build
- Dockerisation
- Déploiement automatique

6.3 Sécurité

- HTTPS (SSL – ACM)
- Variables d'environnement sécurisées
- Backups automatiques
- Rate limiting

---

7. Performance & qualité

- Lazy loading
- Pagination
- Cache (Redis – optionnel)
- Tests unitaires et e 2 e
- Monitoring (CloudWatch)

---

8. UX/UI

- Design responsive
- Interface claire et moderne
- Dark / Light mode
- Accessibilité (WCAG basique)

---

9. Livrables

- Code source (GitHub)
- Documentation technique
- Documentation utilisateur
- Scripts de déploiement
- Diagrammes (UML, Architecture)

---

10. Planning estimatif

- Analyse & conception : 1 semaine
- Développement frontend : 3 semaines

- Développement backend : 3 semaines
- Tests & sécurité : 1 semaine
- Déploiement AWS : 1 semaine

⏱️ **Durée totale estimée : 9 semaines**

---

11. Intégration IA – Reporting intelligent (Phase 2)

11.1 Objectifs de l'IA

- Fournir des **rapports intelligents automatisés**
- Aider à la **prise de décision** (clients & managers)
- Détecter les retards, risques et goulots d'étranglement
- Améliorer la planification des projets

11.2 Cas d'usage IA

a) Reporting intelligent

- Génération automatique de rapports hebdomadaires / mensuels
- Résumé de l'état des projets (avancement, risques, délais)
- Comparaison prévision vs réel

b) Analyse prédictive

- Prédiction des retards de livraison
- Estimation de la date réelle de fin de projet
- Identification des tâches à risque

c) Analyse de performance

- Évaluation de la charge de travail par membre
- Détection du surmenage ou sous-utilisation
- Recommandations de réaffectation des tâches

d) Assistant IA (optionnel)

- Chat IA interne (type Copilot)
- Questions en langage naturel :
- "Pourquoi ce projet est en retard ?"
- "Quels projets sont à risque cette semaine ?"

11.3 Architecture IA

Données exploitées

- Projets

- Tâches
- Délais
- Historique des statuts
- Activité des utilisateurs

Pipeline IA

1. Collecte des données (PostgreSQL)
2. Nettoyage & agrégation
3. Modèles ML
4. API IA exposée au backend NestJS

11.4 Technologies IA

- **Langage** : Python
- **Frameworks ML** :
- Scikit-learn
- TensorFlow / PyTorch (optionnel)
- **NLP** : OpenAI API / LLM
- **Data processing** : Pandas, NumPy
- **Visualisation** : Chart.js / Recharts

11.5 Déploiement IA (AWS)

- **EC2 / ECS** : service IA
- **S3** : stockage datasets & modèles
- **Lambda** : jobs IA planifiés
- **CloudWatch** : monitoring

11.6 Sécurité & conformité

- Anonymisation des données sensibles
- Accès IA par rôles
- Logs & audit

11.7 Évolutions futures IA

- Auto-priorisation des tâches
- Recommandations stratégiques
- Benchmark multi-projets

Auteur : Expert Full Stack & DevOps AWS